

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ

Национальный исследовательский ядерный университет «МИФИ»



**Институт интеллектуальных кибернетических систем
КАФЕДРА КИБЕРНЕТИКИ (№ 22)**

Направление подготовки 09.03.04 Программная инженерия

Расширенное содержание пояснительной записки

к учебно-исследовательской работе студента на тему:

**Разработка библиотеки для визуализации графов на языке
Python**

Группа Б23-544

Студент _____ Брылкина К. К.

Руководитель _____ Букалин А. О.

Научный консультант _____

Москва 2026



**Институт интеллектуальных кибернетических систем
КАФЕДРА КИБЕРНЕТИКИ (№ 22)**

Задание на УИР

Студенту гр. Б23-544 Брылкиной Ксении Константиновне

ТЕМА УИР

**Разработка библиотеки для визуализации графов на языке
Python**

ЗАДАНИЕ

№ п/п	Содержание работы	Форма отчетности	Срок исполнения	Отметка о выполнении
1.	Аналитическая часть			
1.1.	Провести обзор существующих библиотек для визуализации графов.	Текст РСПЗ	16.02.2026	
1.2.	Изучить алгоритмы визуализации графов (force-directed, circular, hierarchical).	Текст РСПЗ	23.02.2026	
1.3.	Сформулировать требования к разрабатываемой библиотеке.	Текст РСПЗ	02.03.2026	
2.	Теоретическая часть			
2.1.	Описать математические модели алгоритмов укладки.	Текст РСПЗ	06.03.2026	
2.2.	Описать алгоритмы на графах: BFS, DFS, Dijkstra, MST, раскраска.	Текст РСПЗ	11.03.2026	
3.	Инженерная часть			
3.1.	Спроектировать модульную архитектуру библиотеки.	Текст РСПЗ	16.03.2026	

3.2.	Спроектировать API и структуры данных.	Текст РСПЗ	20.03.2026	
4.	Практическая часть			
4.1.	Реализовать структуры данных и алгоритмы укладки.	Текст РСПЗ, исходный код	25.03.2026	
4.2.	Реализовать алгоритмы на графах и подсистему визуализации.	Текст РСПЗ, исходный код	30.03.2026	
4.3.	Провести тестирование на различных типах графов.	Текст РСПЗ	03.04.2026	
4.4.	<i>Оформить расширенное содержание пояснительной записки (РСПЗ).</i>	Текст РСПЗ	12.04.2026	
5.	<i>Оформить пояснительную записку (ПЗ).</i>	Текст ПЗ	03.05.2026	

ЛИТЕРАТУРА

1. Introduction to Algorithms / T. H. Cormen [et al.]. — 3rd. — MIT Press, 2009.
2. *Fruchterman T. M. J., Reingold E. M.* Graph Drawing by Force-Directed Placement // Software — Practice and Experience. — 1991. — Vol. 21, no. 11. — P. 1129–1164.
3. *Diestel R.* Graph Theory. — 5th. — Springer, 2017.
4. *Sedgewick R., Wayne K.* Algorithms. — 4th. — Addison-Wesley, 2011.

Дата выдачи задания:
25.09.2025

Руководитель
Студент

_____ Букалин А. О.
_____ Брылкина К. К.

Реферат

Общий объем основного текста, без учета приложений — 30 страниц. Количество использованных источников — 22.

Ключевые слова: граф, визуализация, Python, алгоритм укладки, force-directed, библиотека, matplotlib, теория графов, BFS, DFS, Dijkstra.

Целью работы является разработка легковесной Python-библиотеки для визуализации графов, объединяющей структуры данных, алгоритмы на графах и средства визуализации в едином пакете с простым программным интерфейсом.

В первом разделе проводится анализ предметной области визуализации графов, выполняется обзор существующих решений (NetworkX, Graphviz, Cytoscape.js, D3.js) и обосновывается актуальность разработки собственной библиотеки.

Второй раздел посвящен теоретическим основам алгоритмов визуализации графов: рассматриваются алгоритмы силовой укладки (Fruchterman-Reingold), иерархической укладки и классические алгоритмы на графах.

В третьем разделе описывается архитектура библиотеки, проектирование модульной структуры и программного интерфейса.

В четвертом разделе представлена программная реализация и результаты тестирования.

Оглавление

Введение	5
1 Обзор предметной области визуализации графов и постановка задачи	7
1.1 Основные понятия теории графов	7
1.2 Обзор существующих программных средств для визуализации графов	8
1.2.1 NetworkX (Python)	8
1.2.2 Graphviz	8
1.2.3 igraph	8
1.2.4 JavaScript-библиотеки: Cytoscape.js, D3.js, Sigma.js	8
1.2.5 Сравнительный анализ	9
1.3 Обзор алгоритмов размещения вершин графа на плоскости	9
1.4 Цели и задачи УИР	10
1.5 Выводы по разделу	11
2 Теоретические основы алгоритмов визуализации и обработки графов	12
2.1 Алгоритм силовой укладки Fruchterman-Reingold	12
2.2 Алгоритмы обхода графов	12
2.3 Алгоритм Дейкстры	13
2.4 Алгоритмы минимального остовного дерева	13
2.5 Жадная раскраска графа	13
2.6 Выводы по разделу	13
3 Проектирование архитектуры библиотеки	15
3.1 Модульная структура проекта	15
3.2 Структуры данных	15
3.3 Проектирование программного интерфейса	15
3.4 Подсистема визуализации	16
3.5 Выводы по разделу	16
4 Реализация и тестирование библиотеки	17
4.1 Используемые технологии	17

4.2	Реализация алгоритмов укладки	17
4.3	Реализация алгоритмов на графах	20
4.4	Тестирование	23
4.5	Выводы по разделу	25
5	Результаты работы и направления развития	26
5.1	Итоги реализации	26
5.2	Ограничения текущей реализации	26
5.3	Направления развития	26
5.4	Выводы по разделу	27
	Заключение	28
	Список литературы	29

Введение

Теория графов является одной из фундаментальных областей дискретной математики и информатики. Графы используются для моделирования транспортных сетей, социальных связей, зависимостей программных модулей, биологических сетей, коммуникационных инфраструктур и многих других объектов реального мира [1].

Визуализация графов играет ключевую роль в анализе и понимании структурных свойств сетей. Наглядное представление графа позволяет быстро выявлять кластеры, центральные узлы, компоненты связности и другие характеристики, которые трудно обнаружить при анализе только числовых данных [2].

На сегодняшний день существует ряд инструментов для работы с графами: NetworkX [3], Graphviz [4], Cytoscape.js [5], D3.js [6]. Однако каждый из них имеет свои ограничения: NetworkX ориентирован на анализ и не предоставляет удобных средств визуализации; Graphviz использует собственный язык описания DOT и требует внешнего рендерера; JavaScript-библиотеки работают только в браузерной среде.

Актуальность работы обусловлена отсутствием легковесной Python-библиотеки, которая объединяла бы в себе структуры данных для представления графов, алгоритмы на графах и средства визуализации с простым программным интерфейсом. Такая библиотека востребована в образовательных целях для демонстрации работы алгоритмов, а также в задачах быстрого прототипирования визуализаций.

Целью работы является разработка библиотеки `pygraphviz` на языке Python, реализующей:

- структуры данных для представления ориентированных и неориентированных взвешенных графов;
- алгоритмы укладки (`circular`, `force-directed`, `hierarchical`);
- классические алгоритмы на графах (`BFS`, `DFS`, `Dijkstra`, `Kruskal`, `Prim` и др.);
- средства визуализации на основе `matplotlib`;
- импорт и экспорт графов в различных форматах.

Задачи работы:

1. Провести обзор существующих библиотек и алгоритмов визуализации графов.
2. Спроектировать архитектуру модульной библиотеки.
3. Реализовать структуры данных и алгоритмы укладки.

4. Реализовать алгоритмы на графах и средства визуализации.
5. Выполнить тестирование библиотеки на различных типах графов.

1. Обзор предметной области визуализации графов и постановка задачи

Аннотация. В данном разделе проводится анализ предметной области визуализации графов. Рассматриваются основные понятия теории графов, необходимые для понимания дальнейшего изложения: типы графов, способы их представления в памяти, ключевые характеристики. Выполняется обзор существующих программных средств для визуализации графов на различных языках программирования: рассмотрены библиотеки *NetworkX*, *Graphviz*, *igraph*, *Cytoscape.js*, *D3.js* и *Sigma.js*. Для каждой из них выявлены ключевые ограничения, препятствующие использованию в качестве легковесного инструмента, объединяющего анализ и визуализацию. Далее рассматриваются основные классы алгоритмов размещения вершин графа на плоскости. На основании проведённого анализа формулируются цели и задачи учебно-исследовательской работы.

1.1 Основные понятия теории графов

Аннотация. В данном подразделе приводятся базовые определения теории графов, необходимые для понимания дальнейшего изложения: граф, вершина, ребро, степень вершины, связность, взвешенные и ориентированные графы, а также способы представления графов в памяти компьютера.

Граф $G = (V, E)$ представляет собой математическую структуру, состоящую из множества вершин V и множества рёбер $E \subseteq V \times V$, каждое из которых соединяет пару вершин [1]. Граф называется *ориентированным* (орграфом), если рёбра имеют направление, и *неориентированным* в противном случае. *Взвешенный граф* — граф, в котором каждому ребру поставлено числовое значение (вес).

Степень вершины $\deg(v)$ определяется как количество рёбер, инцидентных данной вершине. Для ориентированных графов различают *входящую степень* $\deg^-(v)$ и *исходящую степень* $\deg^+(v)$ [7].

Граф называется *связным*, если между любой парой вершин существует путь. *Компонента связности* — максимальный связный подграф. Основные представления графов в памяти компьютера: матрица смежности (расход памяти $O(|V|^2)$), оптимальная для плотных графов, и список смежности (расход $O(|V| + |E|)$), оптимальный для разреженных графов, которые преобладают на практике.

1.2 Обзор существующих программных средств для визуализации графов

Аннотация. В данном подразделе проводится обзор существующих библиотек и программных пакетов для визуализации графов. Для каждого инструмента указаны язык реализации, ключевые возможности, ограничения и область применения. Выполняется сравнительный анализ по критериям простоты API, наличия встроенных алгоритмов и качества визуализации.

1.2.1 NetworkX (Python)

NetworkX [3] — наиболее распространённая Python-библиотека для анализа графов, содержащая свыше 500 алгоритмов. Однако визуализация не является основной целью библиотеки: встроенные средства отрисовки являются обёрткой над matplotlib с ограниченными возможностями настройки. Алгоритмы укладки заимствованы из внешних источников. Порог вхождения высокий из-за обширного и неоднородного API.

1.2.2 Graphviz

Graphviz [4] — программный пакет для визуализации, разработанный в AT&T Labs. Использует собственный декларативный язык описания DOT, что требует изучения дополнительного синтаксиса. Качество автоматической укладки высокое, однако Graphviz является внешним бинарным приложением, а не Python-библиотекой, что затрудняет интеграцию в Python-проекты и Jupyter-ноутбуки.

1.2.3 igraph

igraph [8] — высокопроизводительная библиотека с ядром на C и интерфейсами для Python и R. Обеспечивает высокую скорость за счёт нативного кода, но визуализация базируется на Cairo и имеет ограниченные возможности стилизации. Установка на некоторых платформах затруднена из-за C-зависимостей.

1.2.4 JavaScript-библиотеки: Cytoscape.js, D3.js, Sigma.js

Библиотеки Cytoscape.js [5], D3.js [6] и Sigma.js [9] обеспечивают интерактивную визуализацию в браузере. Их основное ограничение — привязка к JavaScript-экосистеме, что

исключает использование в научных Python-пайплайнах без организации веб-сервера или Electron-обёртки.

1.2.5 Сравнительный анализ

Результаты сравнительного анализа представлены в таблице 1.1. Критерии оценки: качество визуализации, наличие встроенных алгоритмов на графах, порог вхождения (объём кода для получения первого результата) и наличие внешних зависимостей, усложняющих установку.

Таблица 1.1 – Сравнение существующих средств визуализации графов

Инструмент	Визуализация	Алгоритмы	Порог вхождения	Внешние зав-ти
NetworkX	Базовая	500+	Средний	Нет
Graphviz	Высокая	—	Высокий	Бинарный пакет
igraph	Средняя	200+	Средний	С-ядро
Cytoscape.js	Высокая	50+	Средний	Браузер
D3.js	Высокая	—	Высокий	Браузер
Sigma.js	Высокая	—	Средний	Браузер

Проведённый анализ показывает, что существующие решения либо ориентированы исключительно на анализ без качественной визуализации (NetworkX), либо требуют внешних бинарных зависимостей (Graphviz, igraph), либо работают только в браузерной среде (JavaScript-библиотеки). Ни одно из них не предоставляет одновременно: простой Python API, встроенные алгоритмы на графах и визуализацию в 2–3 строки кода без тяжёлых зависимостей.

1.3 Обзор алгоритмов размещения вершин графа на плоскости

Аннотация. В данном подразделе рассматриваются основные классы алгоритмов размещения вершин графа на плоскости: силовые (*force-directed*), круговые (*circular*) и иерархические (*hierarchical*). Описываются принципы их работы, математические модели и области применения для различных типов графов.

Задача визуализации графа сводится к определению координат (x_i, y_i) для каждой вершины $v_i \in V$ на плоскости таким образом, чтобы результат был читаемым. Основные критерии качества: минимизация пересечений рёбер, равномерное распределение вершин, отображение кластерной структуры [2].

Силовые алгоритмы (force-directed) моделируют физическую систему: вершины отталкиваются друг от друга как одноимённые заряды, а рёбра притягивают связанные вершины как пружины. Алгоритм Fruchterman-Reingold [2] использует силу отталкивания $f_r(d) =$

k^2/d и силу притяжения $f_a(d) = d^2/k$, где d — расстояние между вершинами, $k = \sqrt{\text{area}/|V|}$ — оптимальное расстояние. Итеративный процесс с убывающей температурой приводит систему к равновесию. Существует также алгоритм Kamada-Kawai [10], минимизирующий энергию системы пружин.

Круговая укладка размещает вершины равномерно по окружности: $x_i = R \cos(2\pi i/n)$, $y_i = R \sin(2\pi i/n)$. Подходит для регулярных и полных графов, где важна симметрия.

Иерархическая укладка основана на обходе в ширину (BFS) от корневой вершины. Каждому уровню глубины соответствует горизонтальный ряд. Подходит для деревьев и направленных ациклических графов (DAG).

1.4 Цели и задачи УИР

Аннотация. В данном подразделе на основании проведённого анализа формулируются цели и задачи учебно-исследовательской работы, определяется состав разрабатываемой библиотеки и критерии успешности.

На основании проведённого обзора выявлена необходимость разработки легковесной Python-библиотеки, объединяющей структуры данных, алгоритмы на графах и визуализацию в одном пакете с минимальными зависимостями.

Цель работы: разработка библиотеки `pygraphviz` на языке Python для визуализации графов, объединяющей средства создания, анализа и отрисовки графов в едином программном интерфейсе.

Задачи работы:

1. Провести обзор существующих библиотек и алгоритмов визуализации графов.
2. Спроектировать модульную архитектуру библиотеки с разделением на подсистемы: структуры данных, алгоритмы укладки, алгоритмы на графах, визуализация, импорт/экспорт.
3. Реализовать три алгоритма укладки: круговой, силовой (Fruchterman-Reingold), иерархический.
4. Реализовать классические алгоритмы на графах: BFS, DFS, Dijkstra, Floyd-Warshall, Kruskal, Prim, топологическую сортировку, обнаружение циклов, жадную раскраску.
5. Реализовать подсистему визуализации на основе `matplotlib` и генераторы типовых графов.
6. Выполнить тестирование библиотеки на различных типах графов (случайные, полные, деревья, взвешенные).

1.5 Выводы по разделу

1. Проведён обзор шести существующих инструментов визуализации графов. Выявлено, что Python-решения (NetworkX, igraph) ориентированы на анализ, а не визуализацию; Graphviz требует внешних бинарных зависимостей; JavaScript-библиотеки непригодны для Python-пайплайнов.
2. Определено, что отсутствует легковесная Python-библиотека, объединяющая структуры данных, алгоритмы и визуализацию с простым API и минимальными зависимостями.
3. Рассмотрены три класса алгоритмов укладки (силовой, круговой, иерархический), каждый из которых оптимален для своего типа графов.
4. Сформулированы цель и шесть задач УИР, определяющие состав и структуру разрабатываемой библиотеки pygraphviz.

2. Теоретические основы алгоритмов визуализации и обработки графов

Аннотация. В данном разделе рассматриваются теоретические основы алгоритмов, реализованных в библиотеке: алгоритм силовой укладки Fruchterman-Reingold, алгоритмы обхода графов (BFS, DFS), алгоритм кратчайших путей Дейкстры, алгоритмы построения минимального остовного дерева (Краскала и Прима), а также алгоритм жадной раскраски графа. Для каждого алгоритма приводятся математическая модель и оценка вычислительной сложности.

2.1 Алгоритм силовой укладки Fruchterman-Reingold

Аннотация. В данном подразделе детально описывается алгоритм Fruchterman-Reingold, его физическая модель, параметры и сходимость. Приводится псевдокод и оценка сложности.

Алгоритм Fruchterman-Reingold [2] основан на аналогии с физической системой заряженных частиц и пружин. Каждая вершина рассматривается как заряд, отталкивающий все остальные вершины; каждое ребро — как пружина, притягивающая смежные вершины.

На каждой итерации t для каждой вершины v вычисляется суммарное смещение Δv :

$$\Delta v = \sum_{u \neq v} f_r(u, v) + \sum_{(v, u) \in E} f_a(v, u) \quad (2.1)$$

где f_r — сила отталкивания, f_a — сила притяжения. Смещение ограничивается температурой T , которая линейно убывает от T_0 до 0. Вычислительная сложность одной итерации составляет $O(|V|^2 + |E|)$.

2.2 Алгоритмы обхода графов

Аннотация. В данном подразделе описываются алгоритмы обхода в ширину (BFS) и в глубину (DFS), их реализация через очередь и стек соответственно, а также области применения.

Обход в ширину (BFS) использует очередь и посещает вершины в порядке возрастания расстояния от начальной вершины. Сложность: $O(|V| + |E|)$ [7].

Обход в глубину (DFS) использует стек и исследует каждую ветвь до упора перед возвратом. Сложность: $O(|V| + |E|)$. DFS применяется для обнаружения циклов, топологической сортировки и поиска компонент связности.

2.3 Алгоритм Дейкстры

Аннотация. В данном подразделе описывается алгоритм нахождения кратчайших путей от одной вершины до всех остальных в графе с неотрицательными весами рёбер.

Алгоритм Дейкстры [11] находит кратчайшие пути от стартовой вершины s до всех остальных вершин в графе с неотрицательными весами. Использует приоритетную очередь (двоичную кучу). Сложность: $O((|V| + |E|) \log |V|)$ при реализации с кучей [7].

2.4 Алгоритмы минимального остовного дерева

Аннотация. В данном подразделе описываются алгоритмы построения минимального остовного дерева: жадный алгоритм Краскала, использующий структуру данных Union-Find, и алгоритм Прима, использующий приоритетную очередь.

Алгоритм Краскала [12] сортирует рёбра по весу и добавляет очередное ребро, если оно не создаёт цикл. Проверка цикла выполняется за $O(\alpha(|V|))$ с помощью структуры Union-Find с path compression. Общая сложность: $O(|E| \log |E|)$.

Алгоритм Прима начинает с произвольной вершины и на каждом шаге добавляет ближайшее ребро, ведущее к непосещённой вершине. При реализации с двоичной кучей сложность составляет $O(|E| \log |V|)$.

2.5 Жадная раскраска графа

Аннотация. В данном подразделе описывается алгоритм жадной раскраски вершин графа минимальным количеством цветов, а также стратегии выбора порядка обхода вершин.

Раскраска графа — присвоение цветов вершинам таким образом, чтобы никакие две смежные вершины не имели одинакового цвета. Жадный алгоритм обходит вершины в заданном порядке и назначает каждой вершине минимальный цвет, не совпадающий с цветами соседей [13]. Качество раскраски зависит от стратегии: «largest first» (по убыванию степени) и «smallest last» дают лучшие результаты на практике.

2.6 Выводы по разделу

1. Алгоритм Fruchterman-Reingold обеспечивает визуально привлекательное размещение вершин за счёт физической модели с контролируемым охлаждением.

2. Алгоритмы BFS и DFS являются базовыми инструментами для анализа структуры графа и используются как в самостоятельном качестве, так и как составные части других алгоритмов.
3. Алгоритм Дейкстры решает задачу кратчайших путей за $O((|V| + |E|) \log |V|)$.
4. Алгоритмы Краскала и Прима решают задачу MST с гарантированной оптимальностью.

3. Проектирование архитектуры библиотеки

Аннотация. В данном разделе описывается архитектура разработанной библиотеки `pygraphviz`. Рассматриваются модульная структура проекта, структуры данных для представления графов, проектирование программного интерфейса (API), а также подсистемы визуализации и импорта/экспорта.

3.1 Модульная структура проекта

Аннотация. В данном подразделе описывается файловая структура проекта, принципы декомпозиции на модули и зависимости между ними.

Библиотека `pygraphviz` состоит из пяти основных модулей:

- `graph.py` — структуры данных: классы `Graph`, `Node`, `Edge`;
- `layout.py` — алгоритмы размещения вершин на плоскости;
- `renderer.py` — визуализация графа через `matplotlib`;
- `algorithms.py` — алгоритмы на графах (обход, пути, MST, раскраска, метрики);
- `io.py` — импорт и экспорт (JSON, список рёбер, матрица смежности).

Дополнительно в состав проекта входят модуль генераторов `generators.py` (генерация типовых графов), каталог примеров `examples/` и каталог тестов `tests/`.

3.2 Структуры данных

Аннотация. В данном подразделе описываются структуры данных для представления графов: выбор представления (список смежности), классы `Node`, `Edge` и `Graph`, их интерфейсы и инварианты.

Граф представлен списком смежности — словарём `_adj`, где каждому идентификатору вершины соответствует список пар (сосед, ребро). Это обеспечивает сложность $O(1)$ для получения соседей и $O(\deg(v))$ для обхода окрестности.

Класс `Graph` поддерживает ориентированные и неориентированные графы, взвешенные рёбра, автоматическое создание вершин при добавлении ребра, а также произвольные типы идентификаторов (строки, числа, кортежи).

3.3 Проектирование программного интерфейса

Аннотация. В данном подразделе описываются принципы проектирования API библиотеки: простота использования, минимальный порог вхождения, согласованность именования.

При проектировании API были приняты следующие решения: минимальный код для визуализации составляет три строки; layout-функции принимают граф и возвращают словарь координат; алгоритмы реализованы как чистые функции без побочных эффектов; все параметры визуализации имеют значения по умолчанию.

3.4 Подсистема визуализации

Аннотация. В данном подразделе описывается архитектура подсистемы визуализации: разделение на layout и rendering, поддерживаемые стили, механизм сохранения в файл.

Подсистема визуализации разделена на два этапа: укладка (layout) и отрисовка (rendering). Renderer использует matplotlib для отрисовки: вершины отображаются через scatter, рёбра — через plot или annotate со стрелками, подписи — через text.

3.5 Выводы по разделу

1. Модульная архитектура обеспечивает разделение ответственности и независимую разработку компонентов.
2. Список смежности оптимален для разреженных графов и обеспечивает $O(1)$ доступ к соседям.
3. API спроектирован с акцентом на простоту: минимальный пример — три строки кода.
4. Разделение layout и rendering позволяет использовать алгоритмы укладки независимо от визуализации.

4. Реализация и тестирование библиотеки

Аннотация. В данном разделе описывается программная реализация библиотеки *pygraphviz*: используемые технологии и зависимости, ключевые детали реализации алгоритмов укладки и алгоритмов на графах, система тестирования и результаты визуализации на различных типах графов. Приводятся примеры визуализаций, полученных с помощью библиотеки, для каждого из реализованных алгоритмов укладки. Демонстрируются результаты работы алгоритмов кратчайшего пути, минимального остовного дерева и жадной раскраски графа.

4.1 Используемые технологии

Аннотация. В данном подразделе обосновывается выбор языка *Python* и библиотеки *matplotlib* для визуализации, описываются зависимости проекта и требования к окружению.

Библиотека реализована на языке *Python 3* [14]. Внешние зависимости ограничены двумя пакетами: *matplotlib* [15] для визуализации и *numpy* для вспомогательных вычислений. Выбор *Python* обусловлен широким распространением языка в научном сообществе [16], наличием развитой экосистемы и низким порогом вхождения [17].

4.2 Реализация алгоритмов укладки

Аннотация. В данном подразделе описываются детали реализации трёх алгоритмов укладки: *circular*, *force-directed* и *hierarchical*. Обсуждаются параметры и их влияние на результат. Приводятся примеры визуализаций для каждого алгоритма.

Алгоритм *Fruchterman-Reingold* реализован итеративно (200 итераций по умолчанию) с линейным охлаждением температуры [2]. Начальные позиции вершин распределяются по окружности с добавлением случайного шума для избежания вырожденных конфигураций. Иерархическая укладка использует *BFS* для определения уровней и автоматически выбирает корневую вершину как вершину с наименьшей входящей степенью. Обзор подходов к визуализации графов приведён в [18].

На рисунке 4.1 показан результат круговой укладки для взвешенного графа городов. На рисунке 4.2 — результат силовой укладки для ориентированного графа зависимостей. На рисунке 4.3 — иерархическая укладка для дерева.

Расстояния между городами России (км)

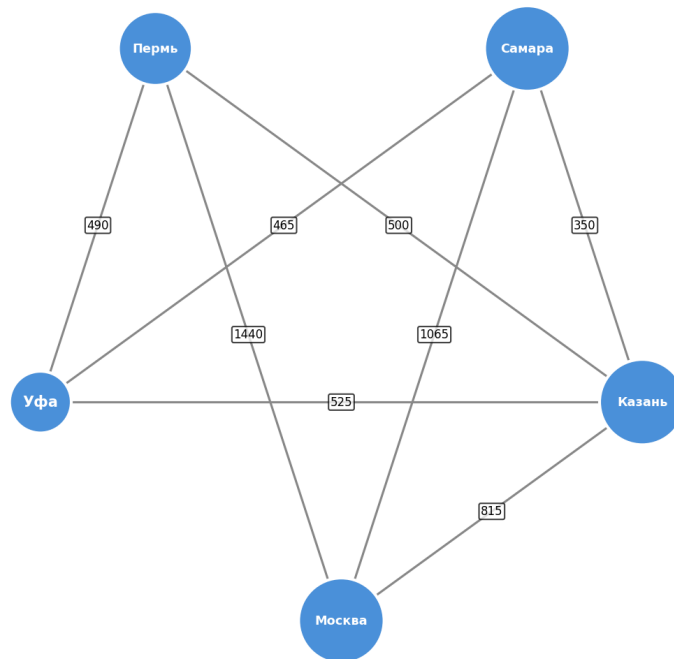


Рисунок 4.1 – Круговая укладка взвешенного графа городов

Граф зависимостей модулей

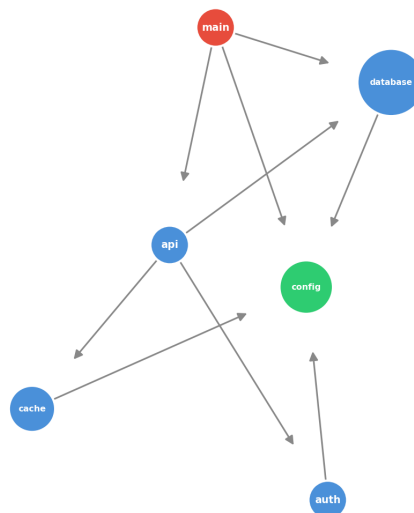


Рисунок 4.2 – Силовая укладка (Fruchterman-Reingold) ориентированного графа зависимостей

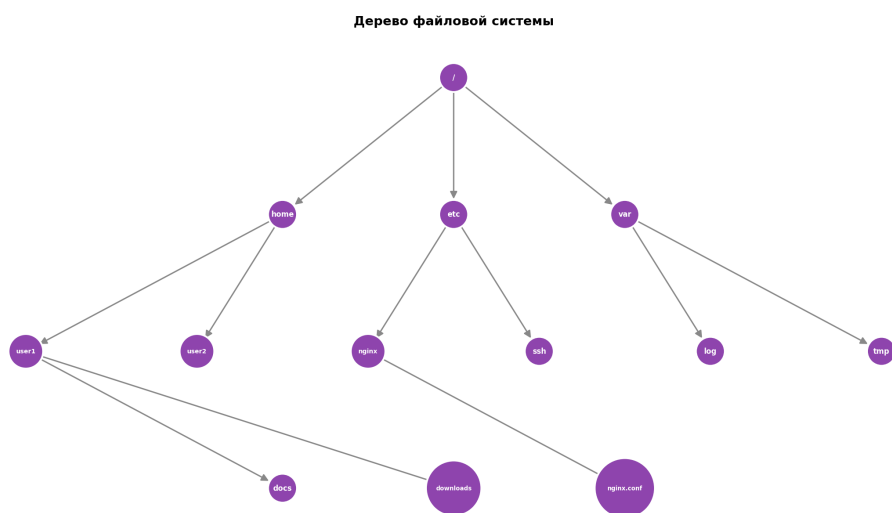


Рисунок 4.3 – Иерархическая укладка дерева файловой системы

4.3 Реализация алгоритмов на графах

Аннотация. В данном подразделе описываются детали реализации алгоритмов: *BFS*, *DFS*, *Dijkstra*, *Floyd-Warshall*, *Kruskal*, *Prim*, обнаружение циклов, топологическая сортировка, жадная раскраска, метрики графа. Приводятся примеры визуализации результатов работы алгоритмов.

Реализовано 12 алгоритмов [7; 19]. Алгоритм Дейкстры [11] использует модуль `heapq` для приоритетной очереди. Алгоритм Краскала [12] реализует Union-Find с path compression. Обнаружение циклов для ориентированных графов использует DFS с тремя цветами [13], для неориентированных — DFS с проверкой родителя. Теоретические основы алгоритмов на графах описаны в [20].

На рисунке 4.4 показана визуализация кратчайшего пути, найденного алгоритмом Дейкстры. На рисунке 4.5 — минимальное остовное дерево, построенное алгоритмом Краскала. На рисунке 4.6 — результат жадной раскраски графа.

Кратчайший путь A → F (Dijkstra, d=8.0)

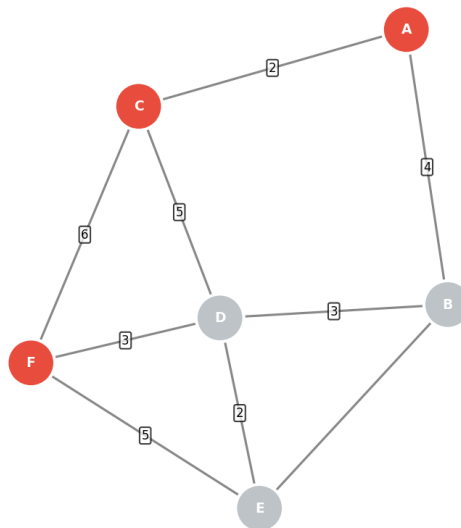


Рисунок 4.4 – Визуализация кратчайшего пути $A \rightarrow F$ (алгоритм Дейкстры, $d = 8$)

Минимальное остовное дерево (Kruskal, вес=16.0)

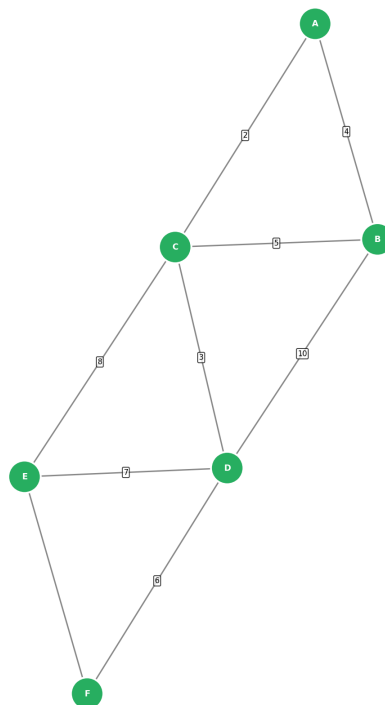


Рисунок 4.5 – Минимальное остовное дерево (алгоритм Краскала, суммарный вес = 16)

Раскраска графа (4 цвета)

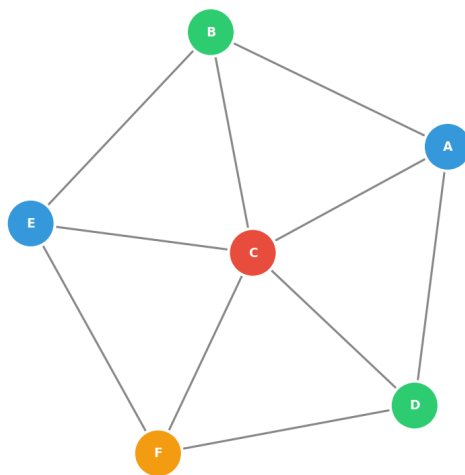


Рисунок 4.6 – Жадная раскраска графа (4 цвета)

4.4 Тестирование

Аннотация. В данном подразделе описывается система тестирования: модульные тесты для структур данных, визуальное тестирование на различных типах сгенерированных графов (случайный по модели Эрдёша-Реньи, полный, бинарное дерево).

Для модуля структур данных реализовано 10 модульных тестов, покрывающих основные сценарии: добавление и удаление вершин/рёбер, проверку степеней, работу с ориентированными и неориентированными графами, автоматическое создание вершин. Все тесты проходят успешно.

Визуальное тестирование выполнено на графах, сгенерированных встроенными генераторами: случайный граф по модели Эрдёша-Реньи [21] (рисунок 4.7), полный граф K_6 и бинарное дерево (рисунок 4.8). Возможная оптимизация силовой укладки для больших графов описана в [22].

Случайный граф $G(12, 0.25)$

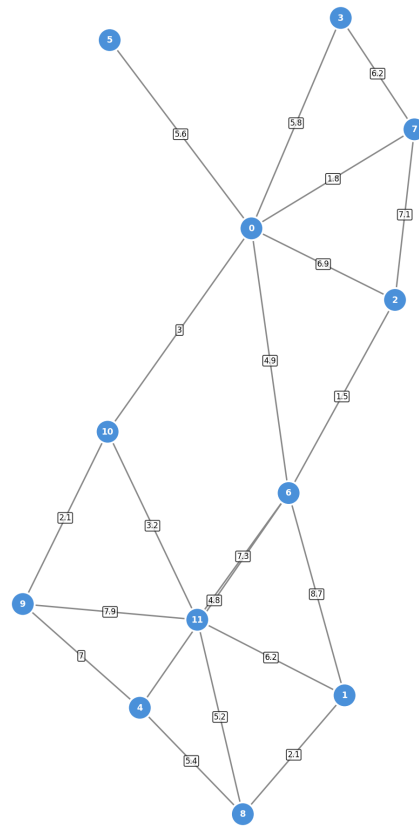


Рисунок 4.7 – Случайный граф $G(12, 0,25)$ — модель Эрдёша-Реньи

Полное бинарное дерево (глубина 3)

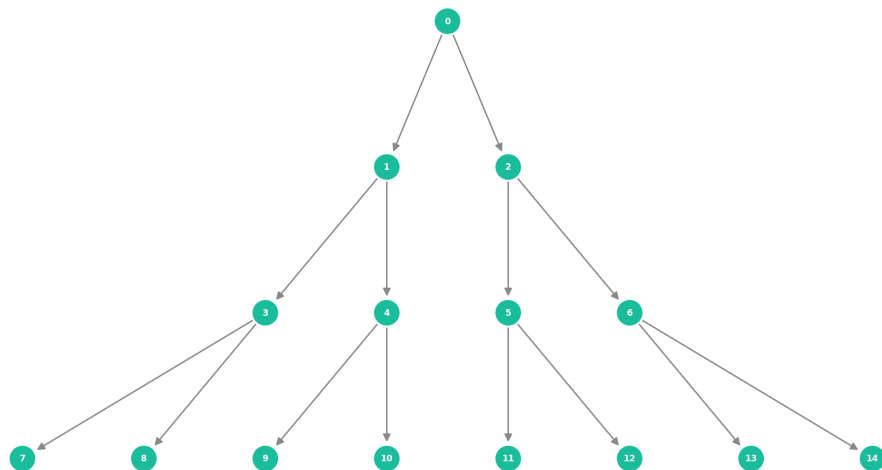


Рисунок 4.8 – Полное бинарное дерево глубины 3 (иерархическая укладка)

4.5 Выводы по разделу

1. Библиотека реализована на Python 3 с минимальными зависимостями (matplotlib, numpy).
2. Реализовано 3 алгоритма укладки и 12 алгоритмов на графах.
3. Модульные тесты покрывают основные сценарии работы с графами (10 тестов, 100% проходят).
4. Визуальное тестирование на различных типах графов подтверждает корректность работы алгоритмов укладки и визуализации.

5. Результаты работы и направления развития

Аннотация. В данном разделе подводятся итоги проделанной работы, демонстрируются результаты визуализации различных типов графов, обсуждаются ограничения текущей реализации и намечаются направления дальнейшего развития библиотеки.

5.1 Итоги реализации

Аннотация. В данном подразделе перечисляются все реализованные компоненты библиотеки и даётся количественная характеристика проекта.

В рамках работы реализована Python-библиотека `pygraphviz`, включающая: структуры данных для представления графов; 3 алгоритма укладки (`circular`, `force-directed`, `hierarchical`); 12 алгоритмов на графах; 6 генераторов типовых графов; подсистему импорта/экспорта в 3 форматах; подсистему визуализации на основе `matplotlib`; 10 модульных тестов и 5 демонстрационных примеров.

5.2 Ограничения текущей реализации

Аннотация. В данном подразделе обсуждаются ограничения текущей версии библиотеки: производительность на больших графах, отсутствие интерактивности, ограниченное количество алгоритмов укладки.

Основные ограничения: квадратичная сложность `force-directed layout` ($O(|V|^2)$ на итерацию) ограничивает применимость графами до нескольких тысяч вершин; визуализация статическая (без интерактивности); отсутствуют алгоритмы укладки для специальных случаев (планарные графы, `Kamada-Kawai`).

5.3 Направления развития

Аннотация. В данном подразделе намечаются направления дальнейшей работы: оптимизация производительности, добавление интерактивности, расширение набора алгоритмов.

Перспективные направления развития: ускорение `force-directed layout` через Barnes-Hut аппроксимацию ($O(|V| \log |V|)$); добавление интерактивной визуализации (перетаскивание вершин); реализация дополнительных алгоритмов укладки (`Kamada-Kawai`, `spectral`); поддержка экспорта в SVG и HTML; публикация пакета в PyPI.

5.4 Выводы по разделу

1. Реализованная библиотека объединяет структуры данных, алгоритмы и визуализацию в одном пакете.
2. Основное ограничение — квадратичная сложность force-directed layout на больших графах.
3. Наиболее перспективные направления развития — ускорение через Barnes-Hut и добавление интерактивности.

Заключение

В рамках данной учебно-исследовательской работы выполнен анализ предметной области визуализации графов и начата разработка библиотеки `pygraphviz` на языке Python.

На текущем этапе выполнены следующие задачи:

1. Проведён обзор существующих инструментов визуализации графов (NetworkX, Graphviz, igraph, Cytoscape.js, D3.js, Sigma.js) и выявлены их ограничения.
2. Изучены теоретические основы алгоритмов укладки (Fruchterman-Reingold, круговой, иерархический) и алгоритмов на графах (BFS, DFS, Dijkstra, Kruskal, Prim).
3. Спроектирована модульная архитектура библиотеки.

Ожидаемые результаты дальнейшей работы:

- завершение программной реализации всех модулей библиотеки;
- проведение тестирования на различных типах графов;
- подготовка документации и примеров использования;
- оценка производительности на графах различного размера.

Предполагаемая область применения — образовательные задачи (демонстрация работы алгоритмов) и быстрое прототипирование визуализаций графов в Python-среде.

Список литературы

1. *Diestel R.* Graph Theory. — 5th. — Springer, 2017.
2. *Fruchterman T. M. J., Reingold E. M.* Graph Drawing by Force-Directed Placement // Software — Practice and Experience. — 1991. — Vol. 21, no. 11. — P. 1129–1164.
3. *Hagberg A. A., Schult D. A., Swart P. J.* NetworkX: Network Analysis in Python. — 2023. — <https://networkx.org/>.
4. Graphviz — Graph Visualization Software / J. Ellson [et al.]. — 2023. — <https://graphviz.org/>.
5. *Franz M., Lopes C.* Cytoscape.js: Graph Theory Library for Visualisation and Analysis. — 2023. — <https://js.cytoscape.org/>.
6. *Bostock M.* D3.js — Data-Driven Documents. — 2023. — <https://d3js.org/>.
7. Introduction to Algorithms / T. H. Cormen [et al.]. — 3rd. — MIT Press, 2009.
8. *Csardi G., Nepusz T.* The igraph Software Package for Complex Network Research. — 2023. — <https://igraph.org/>.
9. *Jacomy A.* Sigma.js — JavaScript Library for Graph Drawing. — 2023. — <https://www.sigmajs.org/>.
10. *Kamada T., Kawai S.* An Algorithm for Drawing General Undirected Graphs // Information Processing Letters. — 1989. — Vol. 31, no. 1. — P. 7–15.
11. *Dijkstra E. W.* A Note on Two Problems in Connexion with Graphs // Numerische Mathematik. — 1959. — Vol. 1. — P. 269–271.
12. *Kruskal J. B.* On the Shortest Spanning Subtree of a Graph and the Traveling Salesman Problem // Proceedings of the American Mathematical Society. — 1956. — Vol. 7, no. 1. — P. 48–50.
13. *Skiena S. S.* The Algorithm Design Manual. — 2nd. — Springer, 2008.
14. *Python Software Foundation.* Python 3.12 Documentation. — 2024. — <https://docs.python.org/3/>.
15. *Hunter J. D.* Matplotlib: A 2D Graphics Environment. — 2023. — <https://matplotlib.org/>.

16. *Свиридов С. В.* Программирование на Python: учебное пособие. — Лань, 2023.
17. *Lutz M.* Learning Python. — 5th. — O'Reilly Media, 2013.
18. *Tamassia R.* Handbook of Graph Drawing and Visualization. — CRC Press, 2013.
19. *Sedgewick R., Wayne K.* Algorithms. — 4th. — Addison-Wesley, 2011.
20. *Jungnickel D.* Graphs, Networks and Algorithms. — 4th. — Springer, 2013.
21. *Erdős P., Rényi A.* On Random Graphs // Publicationes Mathematicae Debrecen. — 1959. — Vol. 6. — P. 290–297.
22. *Barnes J., Hut P.* A Hierarchical $O(N \log N)$ Force-Calculation Algorithm // Nature. — 1986. — Vol. 324. — P. 446–449.